Lab Assignment-1

Name:	**Mohd Aman**
Roll_no:	**2401201145**
Course_Name/Code:	**Operating System Lab/ (ENCA252)**
Program_name:	**BCA (AI & DS)**
Assistant Professor:	**Dr. Sneh Lata Yadav**

Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

Introduction

CPU scheduling is an important function of an operating system. It determines the order in which processes are executed by the CPU. In this assignment, we implement and analyze two scheduling algorithms: First Come First Serve (FCFS) and Shortest Job First (SJF).

Objectives

- Understand CPU scheduling concepts
- Implement FCFS and SJF algorithms
- Calculate CT, TAT, WT
- Compare performance of algorithms

Tools which are used

- Python 3.14.2
- VS Code / Command Promt
- Windows OS

Task 1: Process Creation & Input Handling

Description:-

In this task, we created a Process class with attributes Process ID (PID), Arrival Time (AT), and Burst Time (BT). We took input for multiple processes from the user and stored them in a list. The process details were then displayed in tabular format.

Code:-

```python
# CPU Scheduling: FCFS & SJF (Non-Preemptive)

class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.ct = 0
        self.tat = 0
        self.wt = 0


# -------------------- INPUT --------------------
def take_input():
    processes = []
    n = int(input("Enter number of processes: "))

    for i in range(n):
        print(f"\nEnter details for Process {i+1}")
        pid = input("Process ID: ")
        at = int(input("Arrival Time: "))
        bt = int(input("Burst Time: "))
        processes.append(Process(pid, at, bt))

    return processes


# -------------------- DISPLAY --------------------
def display(processes, title):
    print(f"\n{title}")
    print("PID\tAT\tBT\tCT\tTAT\tWT")

    for p in processes:
        print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")

```

Output:-

```
PS D:\Assignments\OS\OS_Lab_Assignment_1> python Lab_Assignment_1.py
Enter number of processes: 5

Enter details for Process 1
Process ID: P0
Arrival Time: 0
Burst Time: 7

Enter details for Process 2
Process ID: P1
Arrival Time: 2
Burst Time: 4

Enter details for Process 3
Process ID: P2
Arrival Time: 4
Burst Time: 1

Enter details for Process 4
Process ID: P3
Arrival Time: 5
Burst Time: 4

Enter details for Process 5
Process ID: P4
Arrival Time: 6
Burst Time: 6

Initial Process Table:
PID     AT      BT
P0      0       7
P1      2       4
P2      4       1
P3      5       4
P4      6       6
```

Task 2: FCFS Scheduling

Description:-

First Come First Serve (FCFS) is a non-preemptive scheduling algorithm in which the process that arrives first is executed first. Processes are executed in the order of their arrival time.

Steps:-

1. Processes are sorted based on Arrival Time.
2. Completion Time (CT) is calculated.
3. Turnaround Time (TAT) = CT - AT
4. Waiting Time (WT) = TAT - BT

Code:-

```python
# -------------------- FCFS --------------------
def fcfs(processes):
    processes.sort(key=lambda x: x.at)
    time = 0
    gantt = []

    for p in processes:
        if time < p.at:
            time = p.at

        start = time
        time += p.bt

        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt

        gantt.append((p.pid, start, time))

    return gantt

```

Output:-

```
FCFS Result
PID      AT       BT       CT       TAT      WT
P0       0        7        7        7        0
P1       2        4        11       9        5
P2       4        1        12       8        7
P3       5        4        16       11       7
P4       6        6        22       16       10

FCFS Gantt Chart:
| P0 | P1 | P2 | P3 | P4 |
0    7    11   12   16   22

Average Waiting Time = 5.80
Average Turnaround Time = 10.20
```

Task 3: SJF Scheduling

Description:-

Shortest Job First (SJF) is a scheduling algorithm in which the process with the smallest burst time is executed first. Only the processes that have already arrived are considered.

Steps:-

1. Select processes that have arrived.
2. Choose the process with the shortest burst time.
3. Execute it and calculate CT, TAT, WT
4. Repeat until all processes are completed.

Code:-

```python
# CPU Scheduling: FCFS & SJF (Non-Preemptive)

class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.ct = 0
        self.tat = 0
        self.wt = 0


# -------------------- INPUT --------------------
def take_input():
    processes = []
    n = int(input("Enter number of processes: "))

    for i in range(n):
        print(f"\nEnter details for Process {i+1}")
        pid = input("Process ID: ")
        at = int(input("Arrival Time: "))
        bt = int(input("Burst Time: "))
        processes.append(Process(pid, at, bt))

    return processes


# -------------------- DISPLAY --------------------
def display(processes, title):
    print(f"\n{title}")
    print("PID\tAT\tBT\tCT\tTAT\tWT")

    for p in processes:
        print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")

```

Output:-

```
SJF Result
PID      AT       BT       CT       TAT      WT
P0       0        7        7        7        0
P1       2        4        12       10       6
P2       4        1        8        4        3
P3       5        4        16       11       7
P4       6        6        22       16       10

SJF Gantt Chart:
| P0 | P2 | P1 | P3 | P4 |
0    7    8    12    16    22

Average Waiting Time = 5.20
Average Turnaround Time = 9.60
```

Task 4: Gantt Chart Representation

Description:-

Gantt chart is used to represent the execution order of processes over time. It helps in visualizing how CPU is allocated to different processes.

Code:-

```python
# -------------------- GANTT CHART --------------------
def print_gantt(gantt, title):
    print(f"\n{title} Gantt Chart:")

    for pid, start, end in gantt:
        print(f"| {pid} ", end="")
    print("|")

    print("0", end="")
    for pid, start, end in gantt:
        print(f"    {end}", end="")
    print()


```

Output:-

```
FCFS Gantt Chart:
| P0 | P1 | P2 | P3 | P4 |
0    7    11    12    16    22
```

```
SJF Gantt Chart:
| P0 | P2 | P1 | P3 | P4 |
0    7    8    12    16    22
```

Task 5: Performance Analysis & Comparison

Description:-

The performance of scheduling algorithms is evaluated using Average Waiting Time and Average Turnaround Time.

From the results, SJF gives better performance as it minimizes the average waiting time compared to FCFS.

Therefore, SJF is more efficient than FCFS.

Code:-

```
# -------------------- COMPARISON --------------------
print("\n========== Comparison ==========")

avg_wt_fcfs = total_wt / len(fcfs)
avg_wt_sjf = total_wt_sjf / len(sjf)
 avg_tat_fcfs = total_tat / len(fcfs)
avg_tat_sjf = total_tat_sjf /
len(sjf)

print(f"FCFS Avg WT: {avg_wt_fcfs:.2f}, Avg TAT: {avg_tat_fcfs:.2f}")
print(f"SJF Avg WT: {avg_wt_sjf:.2f}, Avg TAT: {avg_tat_sjf:.2f}")
 if avg_wt_sjf <
avg_wt_fcfs:
    print("SJF is better (less waiting time)")
else:
    print("FCFS is better")
```

Output:-

```
========== Comparison ==========
FCFS Avg WT: 3.33, Avg TAT: 8.67
SJF Avg WT: 3.33, Avg TAT: 8.67
FCFS is better
```

Conclusion:-

In this assignment, we successfully implemented FCFS and SJF scheduling algorithms. We calculated scheduling parameters and compared their performance. This helped in understanding CPU scheduling concepts effectively.